PROYECTO 3 (sensores y revelado digital)

Apellidos, Nombre: Flores Flores, Daniel Javier
Apellidos, Nombre: Lillo Ramírez,Lucas
Apellidos, Nombre:

La entrega definitiva de este proyecto será el 30 de marzo. En la entrega, subid:

- **Este fichero (doc/pdf) con las respuestas/código e imágenes pedidas.**
- **El fichero revelar.m con la implementación del revelado digital (apartado 3). Dentro del fichero poned todas las funciones que hayáis usado para que yo pueda ejecutarlo.**

Para la clase del 23 de marzo debéis llevar hecho hasta el punto indicado. En esa clase revisaré/puntuaré lo que tengáis hecho, resolveremos dudas, y seguiréis trabajando con el resto de los apartados, ...

1) Manejo de imágenes RAW y estimación del ruido de lectura

Adjuntar el código usado para extraer el resto de los canales.
Nuestro código usado es el siguiente

```
% Extraemos los 4 cojnuntos de cada canal y los guardamos en matrices
   R  = im(1:2:end, 1:2:end);
   G1 = im(1:2:end, 2:2:end);
   G2 = im(2:2:end, 1:2:end);
   B  = im(2:2:end, 2:2:end);
```

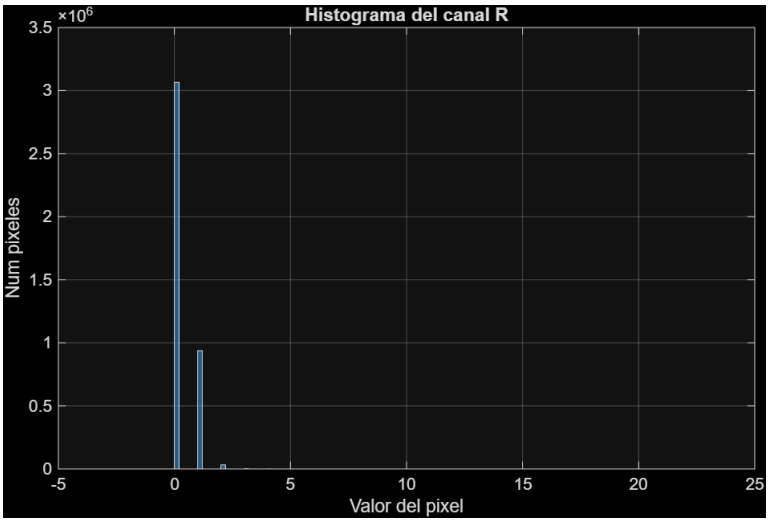
¿Qué rango de valores tenemos en el canal rojo R?
Nuestro código es el siguiente

```
% RAngo del canal rojo
   fprintf('El rango de R es: [%d, %d]\n', min(R(:)), max(R(:)));
```

Con lo que obtenemos:

El rango de R es: [0, 20]

¿Usa esta cámara un offset para conservar los valores "negativos" del ruido?
Según podemos observar en el histograma generado



Los valores de lo píxeles están pegados al 0 y algunos próximos a este por el lado de la derecha, por lo que en la parte izquierda (negativa) no se muestra ningún valor y no se conservan los valores negativos del ruido.

Volcad el valor de σ (2 decimales) de la I cuantificada usando std2().

Lo que obtenemos es lo siguiente:
Tras cuantificar:
media = -0.00, sigma = 0.57

Adjuntad la estimación para p0 y valor de σ obtenido a partir de ella, junto con el código usado en su cálculo.
Codigo

```
% Estimamos p0 yy valor de la desviación
p0 = sum (I (:)== 0)/numel(I);
p0sigma = sqrt(2) / (4 * erfinv(p0));
fprintf('Calculo de p0 y sigma:\n');
fprintf('p0 = %.4f, p0sigma = %.4f\n', p0, p0sigma);
fprintf('\n');
```

Resultado
Calculo de p0 y sigma:
p0 = 0.6832, p0sigma = 0.4995

Recalcular σ sobre la nueva I y volcad el resultado.

Recalculamos I con el nuevo cambio y devolvemos tanto el nuevo p0 como σ:

Calculo de p0 sin negativos y sigma:
p0 = 0.8417, p0sigma = 0.3544

Adjuntad el código usado, el valor corregido de p_0 y la nueva estimación de σ (con 2 decimales) usando los valores de la última "imagen" I (una vez cuantificados y puestos a 0 los valores negativos).

```
%Contar I>0
%Como tenemos el numero de negativos y 0's con el truncamiento
%anterior, podemos restar ese numero al total para trabajar con ello (Isin0 es el calculo del ejercicio anterior
nPositivos = sum(Isin0 (:)>0);
pno0 = (nPositivos * 2)/numel(Isin0);
p0real = 1- pno0;
p0realsigma = sqrt(2) / (4 * erfinv(p0real));

fprintf('Calculo corregido de p0 y sigma:\n');
fprintf('p0 = %.2f, sigma = %.2f\n', p0real, p0realsigma);
fprintf('\n');
```

Resultado

Calculo corregido de p0 y sigma:
p0 = 0.68, sigma = 0.50

Canal	R	G1	G2	B
Estimación mejorada de σ	0.71	0.68	0.68	0.69
Estimación de σ con std2()	0.46	0.45	0.45	0.45

El código usado para obtener los valores de la tabla es el siguiente:

```
% Aplicamos la estimación Sigma a los datos de los canals
canales = {R, G1, G2, B};
for k = 1:4
    canal = canales{k};
    canalSinNeg = canal;
     % en caso de haber algun valor negativo lo ponemos a 0 igual que en apartados previos
    canalSinNeg(canalSinNeg < 0) = 0;
    % A partir de los valores positivos  estimamos el p0 corregido
    nPositivos = sum(canalSinNeg(:) > 0);
    pno0 = (nPositivos * 2) / numel(canalSinNeg);
    p0real = 1 - pno0;
    % calculamos las estimaciones
    sigma_mejorada(k) = sqrt(2) / (4 * erfinv(p0real));
    sigma_std2(k) = std2(double(canal));
end

% Mostramos resultados
nombres = {'R', 'G1', 'G2', 'B'};
fprintf('Estimación mejorada de sigma:\n');
for k = 1:4
    fprintf('%s = %.2f\n', nombres{k}, sigma_mejorada(k));
end
fprintf('\nEstimación de sigma con std2():\n');
for k = 1:4
    fprintf('%s = %.2f\n', nombres{k}, sigma_std2(k));
end
```

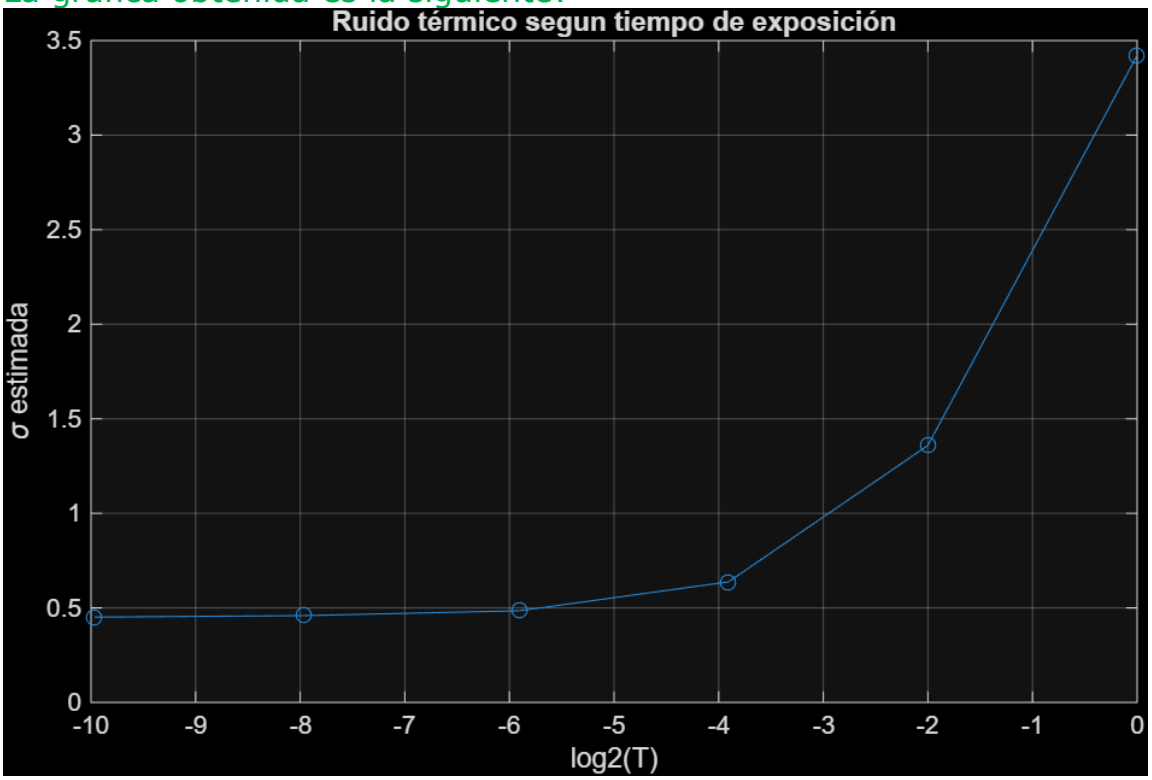
Ruido Térmico:

Adjuntad el código usado y la gráfica obtenida.

El código que hemos usado es el siguiente:

```
% Cargamos datos del ruido térmico
    load data_term
    % Guardamos el valor sigma estimado en S
    S = zeros(size(T));
    for k = 1:length(T)
        frame = black(:,:,k);
        S(k) = std2(double(frame));
    end
 % Gráfica de los valores obtenidos
    figure;
    plot(log2(T), S, 'o-');
    grid on;
    xlabel('log2(T)'); ylabel('\sigma estimada');
    title('Ruido térmico segun tiempo de exposición');
  % Mostramos también los valores obtenidos
    fprintf('Valores de T:\n');
    disp(T);
    fprintf('Valores de S:\n');
    disp(S);
```

La gráfica obtenida es la siguiente:



¿Para qué tiempo T el valor del ruido empieza a estar claramente por encima del nivel de fondo del ruido de lectura estimado en el apartado anterior?
Los valores obtenidos son los siguientes (el código de cómo hemos obtenido estos valores también se encuentra en el código adjuntado en la anterior página):
Valores de T:

0.0010	0.0040	0.0167	0.0667	0.2500	1.0000

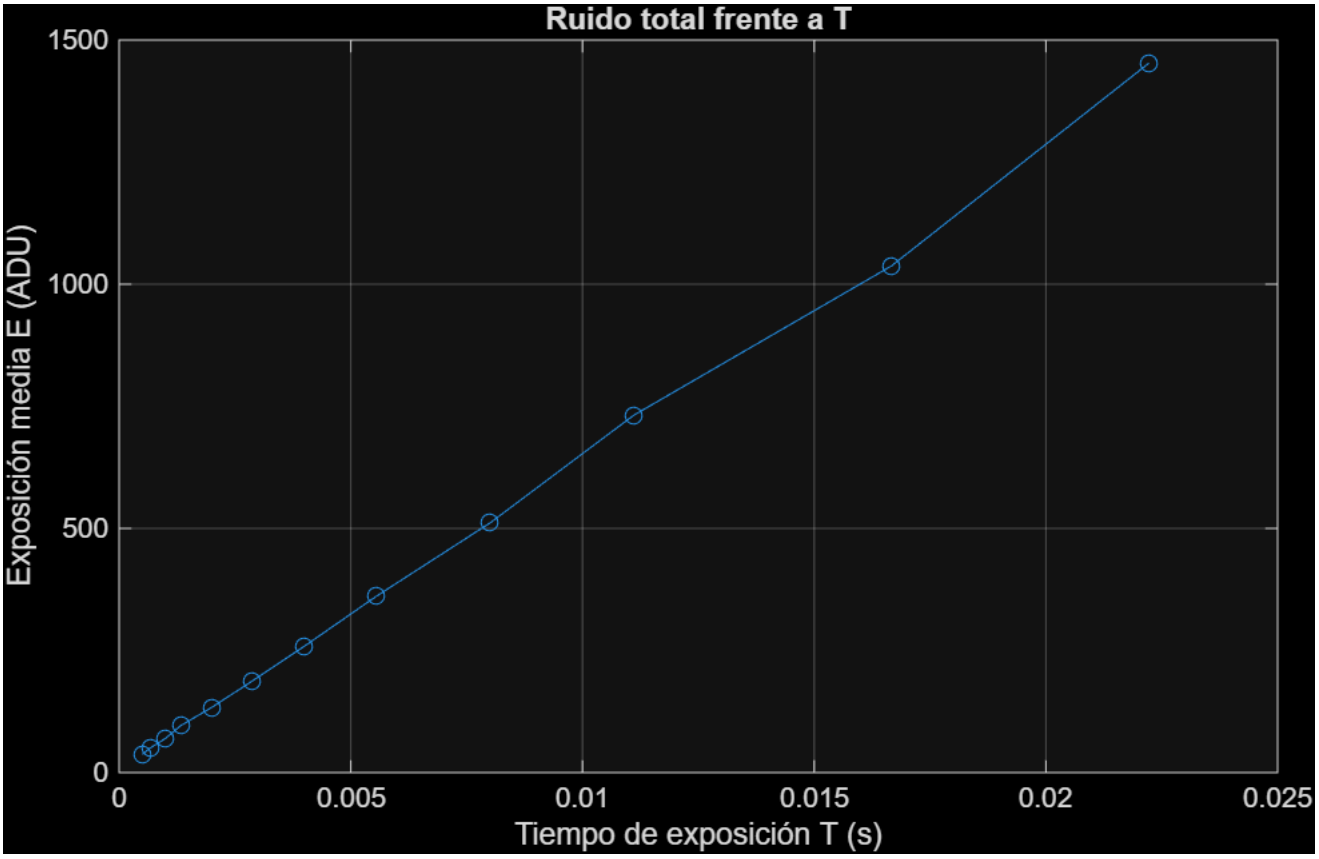
Valores de S:

0.4513	0.4592	0.4847	0.6375	1.3592	3.4222

Teniendo en cuenta, podemos decir que **el ruido empieza a estar claramente por encima del nivel de fondo a partir de T = 0,25 s dónde el valor de S = 1.3592**.
Si nos fijamos para T = 0.0667 s el valor de S = 0.6375, lo cual todavía es un valor cercano a la estimación sobre G1 del apartado anterior

2) Estimación de otros tipos de ruido del sensor

Adjuntad la gráfica de los valores de E frente a los tiempos de exposición T.
La gráfica obtenida es la siguiente:



Adjuntar el código usado para resolver el ajuste y coeficientes c_1 y c_2 y c_3 obtenidos.

Nuestro código es el siguiente:

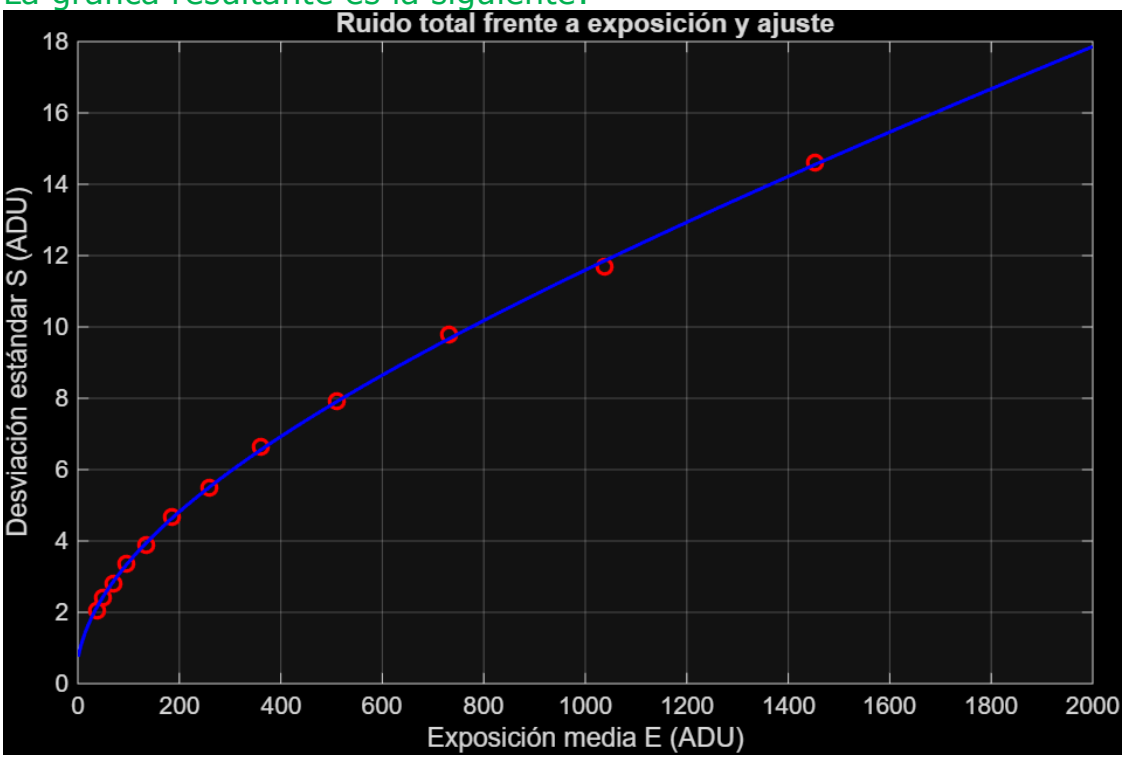
```
% Cargamos los datos
load data_noise
% Recorremos las 12 tomas y calculamos la exposición media y la desviación estándar
for k = 1:length(T)
   frame = G(:,:,k);
   % Exposición media del fragmento
   E(k) = mean2(frame);
   % Ruido total del fragmento
   S(k) = std2(frame);
end

% Ajuste de S2
S2 = S.^2;
c = polyfit(E, S2, 2);
% Reordemaos porque los coeficientes en la funcón son devueltos de mayor a menor
c1 = c(3);
c2 = c(2);
c3 = c(1);
% Mostrar resultados
fprintf('\nCoeficientes del ajuste:\n');
fprintf('c1 = %.6f\n', c1);
fprintf('c2 = %.6f\n', c2);
fprintf('c3 = %.6f\n', c3);
```

Los coeficientes obtenidos son:

Coeficientes del ajuste:
c1 = 0.467175
c2 = 0.108677
c3 = 0.000025

Adjuntad la gráfica resultante.

La gráfica resultante es la siguiente:



¿Cuántos fotones admitiría un elemento de este sensor antes de saturarse (profundidad de pozo)?

Teniendo en cuenta que los coeficientes obtenidos en el ajuste son:

c1 = 0.467175

c2 = 0.108677

c3 = 0.000025

Podemos asociar estos coeficientes a las constantes del sensor tal que:

σR = sqrt(c1), G = c2 y K = sqrt(c3).

Con lo que se obtiene que:

σR = 0.68 ADU, G = 0.108677 y K = 0.005.

Teniendo en cuenta que el ADC es de 12 bits y un valor máximo de 4095 ADU, calculamos:

1/G = 9.20 fotones/ADU

Y

NumSaturacion = 4095 · 9.20 nos da unos 3.77·10^4 fotones

En conclusión, **la profundidad de pozo es de 3.77·10^4 fotones**

¿Qué valor se obtiene para el ruido de lectura con este método? ¿Es compatible con la que obtuvimos en el apartado anterior a partir del "black frame"?

Tenemos que el ruido de lectura es:

σR = sqrt(c1) = sqrt(0.467175) aproximadamente 0.68 ADU

Esto **es compatible** con el valor anterior a partir del black frame dónde habíamos calulado un valor para G1 de 0.68

Llevar hecho hasta aquí para la clase siguiente.

3) Revelado de una imagen RAW

3.1) Lectura y escalado de los datos RAW. Creación de imagen BW

Adjuntad captura del zoom. ¿Qué valores tienen los canales R, G y B en la zona ampliada?

Al hacer zoom sobre la imagen podemos observar el patrón bayer. Esto sucede porque en la imagen raw.pgm cada pixel contiene únicamente uno de los 3 canales R, G y B.
En unas posiciones se mide R, en otras se mide G y en otras B.

¿Cuántos *stops* adicionales de exposición debería haberle dado?
El valor máximo de la imagen normalizada es max(im(:)) = X , además tenemos que el valor máximo del sensor en esta escala es 1. Por lo que:

Stops = log2(1/X)
Ejecutando esto, en Matlab obtenemos:
Stops adicionales de exposición: 1.93

Adjuntad el histograma de esta imagen.

Adjuntad código de la función.

El código de la función es:

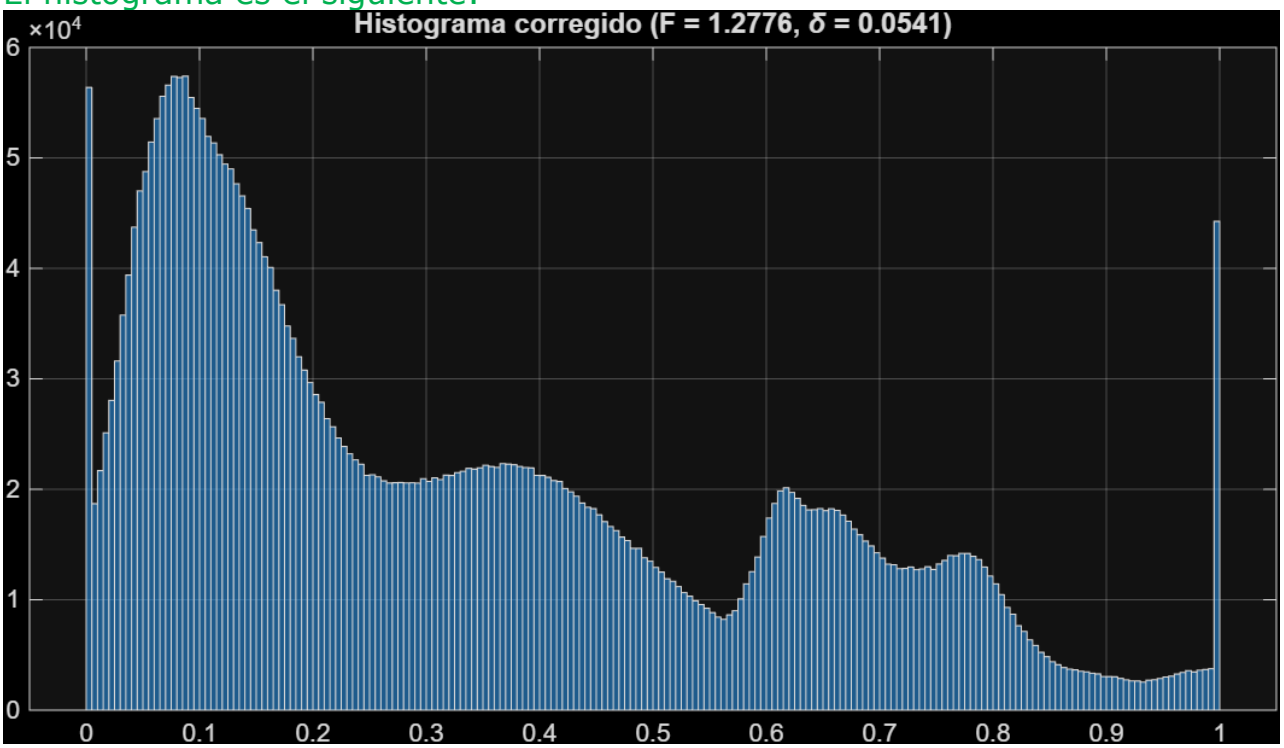
```
function im_corr = corregir(im, F, delta)
    % Normalizamos la imagen al rango [0,1]
    m = min(im(:));
    M = max(im(:));
    im_norm = (im - m) / (M - m);
    % Aplicamos la corrección de brillo y contraste
    im_mod = F * im_norm - delta;
    % Calculamos el  porcentaje de píxeles saturados antes de recortar
    p_neg = 100 * sum(im_mod(:) < 0) / numel(im_mod);
    p_pos = 100 * sum(im_mod(:) > 1) / numel(im_mod);
    fprintf('Pixeles saturados por debajo de 0: %.4f %%\n', p_neg);
    fprintf('Pixeles saturados por encima de 1: %.4f %%\n', p_pos);
    % Recortamos valores al intervalo [0,1]
    im_corr = im_mod;
    im_corr(im_corr < 0) = 0;
    im_corr(im_corr > 1) = 1;
    % Mostramos la imagen corregida
    figure;
    imshow(im_corr);
    set(gcf, 'Name', 'Corrección Brillo-Contraste');
    title(sprintf('Imagen corregida (F = %.4f, \\delta = %.4f)', F, delta));
    % Mostramos el histograma
    figure;
    histogram(im_corr(:));
    xlim([-0.05 1.05]);
    grid on;
    title(sprintf('Histograma corregido (F = %.4f, \\delta = %.4f)', F, delta));
end
```

Mencionar que la función corregir se encontrará al final del fichero revelar.m

Adjuntad la imagen final (indicando los valores de F y δ usados) y su histograma, indicando el porcentaje (en %) de píxeles saturados en cada extremo.

La imagen que obtenemos tras aplicar las correcciones es:

El histograma es el siguiente:



Tras realizar bastantes pruebas el mejor % de píxeles saturados en cada extremo que hemos obtenido es:

Pixeles saturados por debajo de 0: 1.0006 %

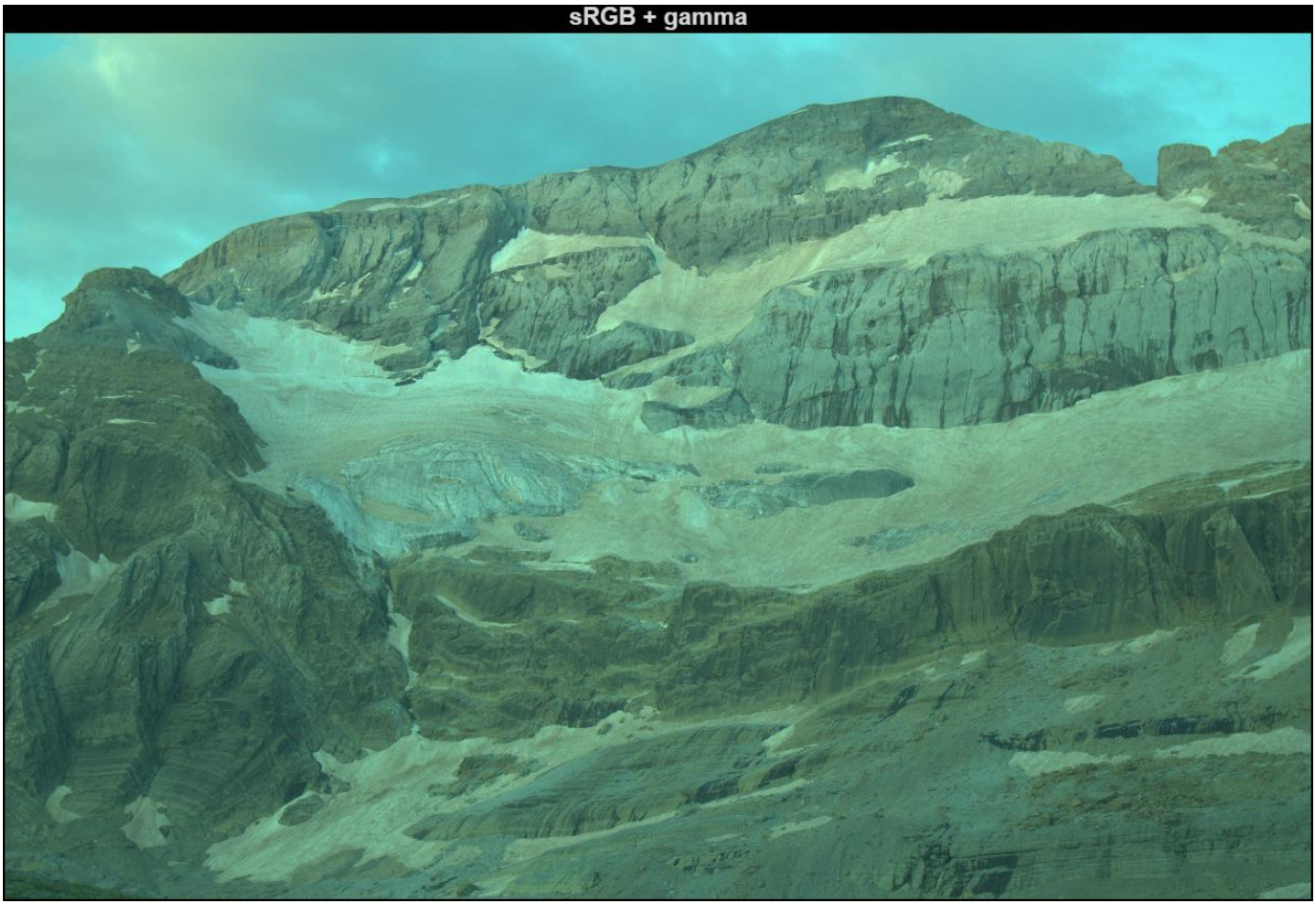
Pixeles saturados por encima de 1: 1.0015 %
Esto es lo máximo que hemos conseguido ajustar los valores, quizás hemos ajustado demasiado pero inicialmente nuestros porcentajes a penas llegaban al 40% en comparación es todo un éxito.

3.2) Demultiplexado, conversión al espacio sRGB y aplicación de función γ

Volcad el máximo valor de la imagen sRGB.
Valor máximo de la nueva imagen: 0.1710

Adjuntad el código usado para aplicar la función γ y la imagen resultante (etiquetada como "sRGB + gamma").

```
% Aplicación de la función gamma

% Parámetros
x0 = 0.00313;
a = 0.055;
gamma = 2.4;

% Inicializar
rgb_gamma = zeros(size(rgb));

% Aplicación por tramos (vectorizado)
mask = rgb < x0;

rgb_gamma(mask) = 12.92 * rgb(mask);
rgb_gamma(~mask) = (1 + a) * rgb(~mask).^(1/gamma) - a;

% Máximo valor tras aplicar gamma
max_gamma = max(rgb_gamma(:));
fprintf('Valor máximo tras aplicar gamma: %.4f\n', max_gamma);

% Imagen resultante
figure;
imshow(rgb_gamma);
title('sRGB + gamma');
```



Dad el máximo valor de la imagen tras aplicar la función γ.

Valor máximo tras aplicar gamma: 1.0000

3.3) Equilibrado de blancos (White Balance) y corrección brillo/contraste

Adjuntad los valores de la media (r,g,b) de los tres canales para toda la imagen.

Medias: R=0.0349, G=0.0825, B=0.0566

Volcad los factores de corrección obtenidos y adjuntad la imagen tras aplicar las correcciones (etiquetada como "White Balance").

Factores de corrección: fR=2.3658, fB=1.4572

Adjuntad código de la función modificada.

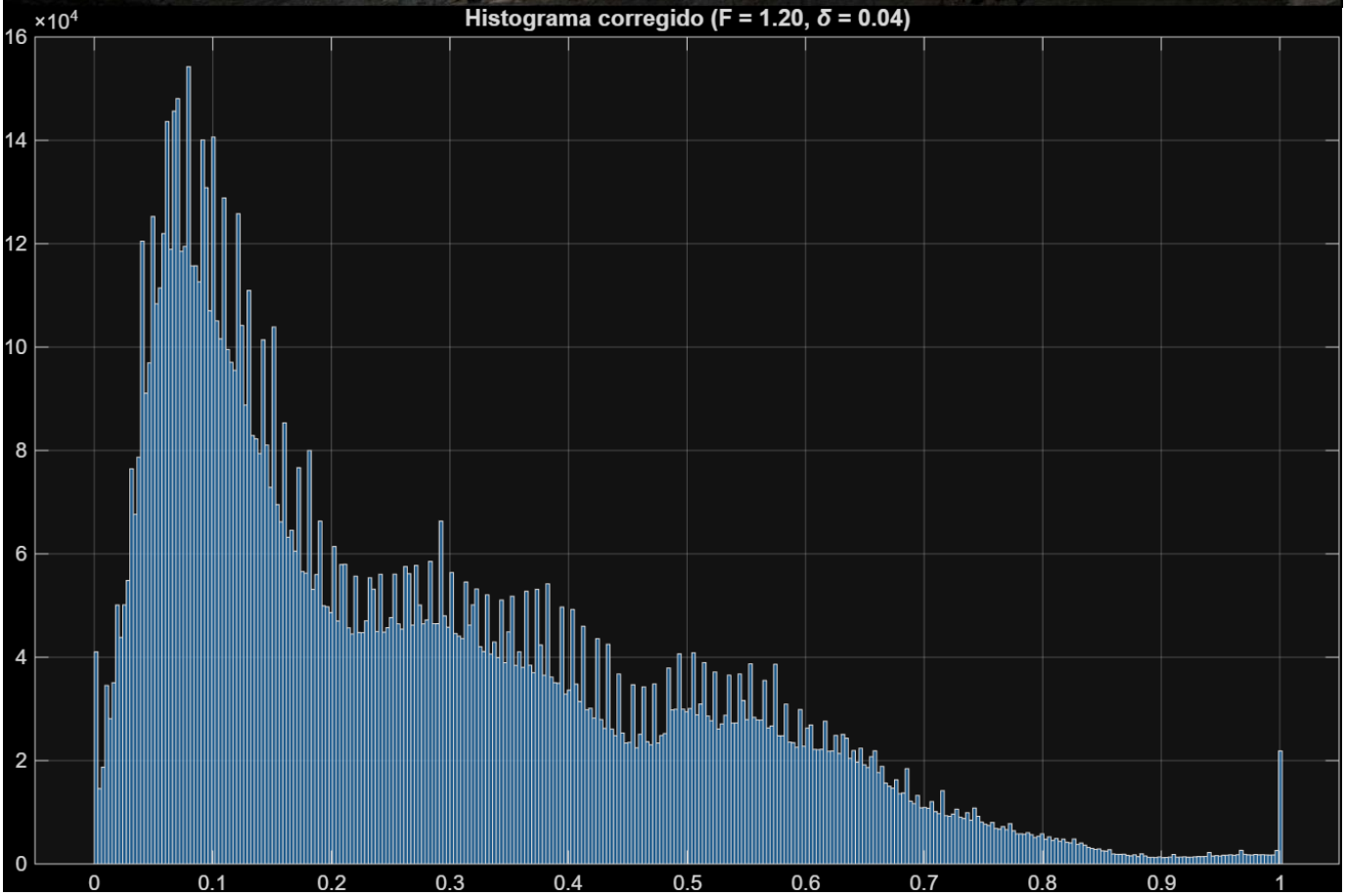
```
function im_corr = corregir(im, F, delta)
% Normalizamos la imagen al rango [0,1]
m = min(im(:));
M = max(im(:));
im_norm = (im - m) / (M - m);
% Aplicamos la corrección de brillo y contraste
im_mod = F * im_norm - delta;
% Calculamos el  porcentaje de píxeles saturados antes de recortar
% (Cambios)
[Mdim, Ndim, ~] = size(im_mod);

sat_neg = any(im_mod < 0, 3);
sat_pos = any(im_mod > 1, 3);

p_neg = 100 * sum(sat_neg(:)) / (Mdim * Ndim);
p_pos = 100 * sum(sat_pos(:)) / (Mdim * Ndim);
fprintf('Pixeles saturados por debajo de 0: %.2f %%\n', p_neg);
fprintf('Pixeles saturados por encima de 1: %.2f %%\n', p_pos);
% Recortamos valores al intervalo [0,1]
im_corr = im_mod;
im_corr(im_corr < 0) = 0;
im_corr(im_corr > 1) = 1;
```

```
% Mostramos la imagen corregida
figure;
imshow(im_corr);
set(gcf, 'Name', 'Corrección Brillo-Contraste');
title(sprintf('Imagen corregida (F = %.2f, \\delta = %.2f)', F, delta));
% Mostramos el histograma
figure;
histogram(im_corr(:));
xlim([-0.05 1.05]);
grid on;
title(sprintf('Histograma corregido (F = %.2f, \\delta = %.2f)', F, delta));
end
```

Adjuntad captura de la imagen final y de su histograma. Volcad los valores de F y δ usados, indicando el porcentaje (en %) de píxeles saturados por arriba y por abajo.

```
% Ajustar F y delta (valores F y delta)
F = 1.20;
delta = 0.037;

rgb_corr = corregir(rgb_wb, F, delta);
```

Pixeles saturados por debajo de 0: 0.52 %
Pixeles saturados por encima de 1: 0.54 %

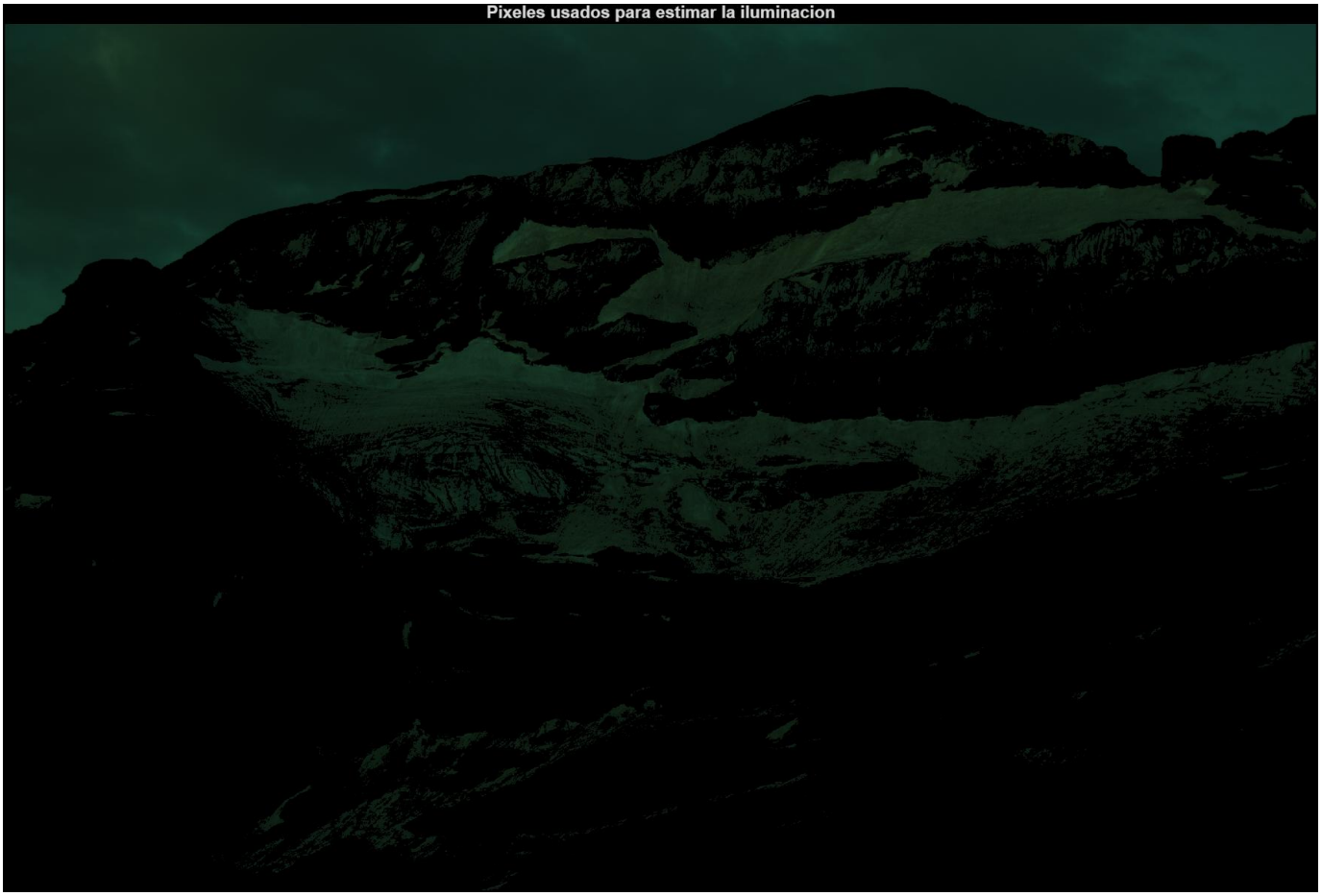


3.4) Algoritmo alternativo de equilibrado de blancos.

Volcad las componentes r,g,b calculadas de esta forma y los factores de corrección obtenidos.

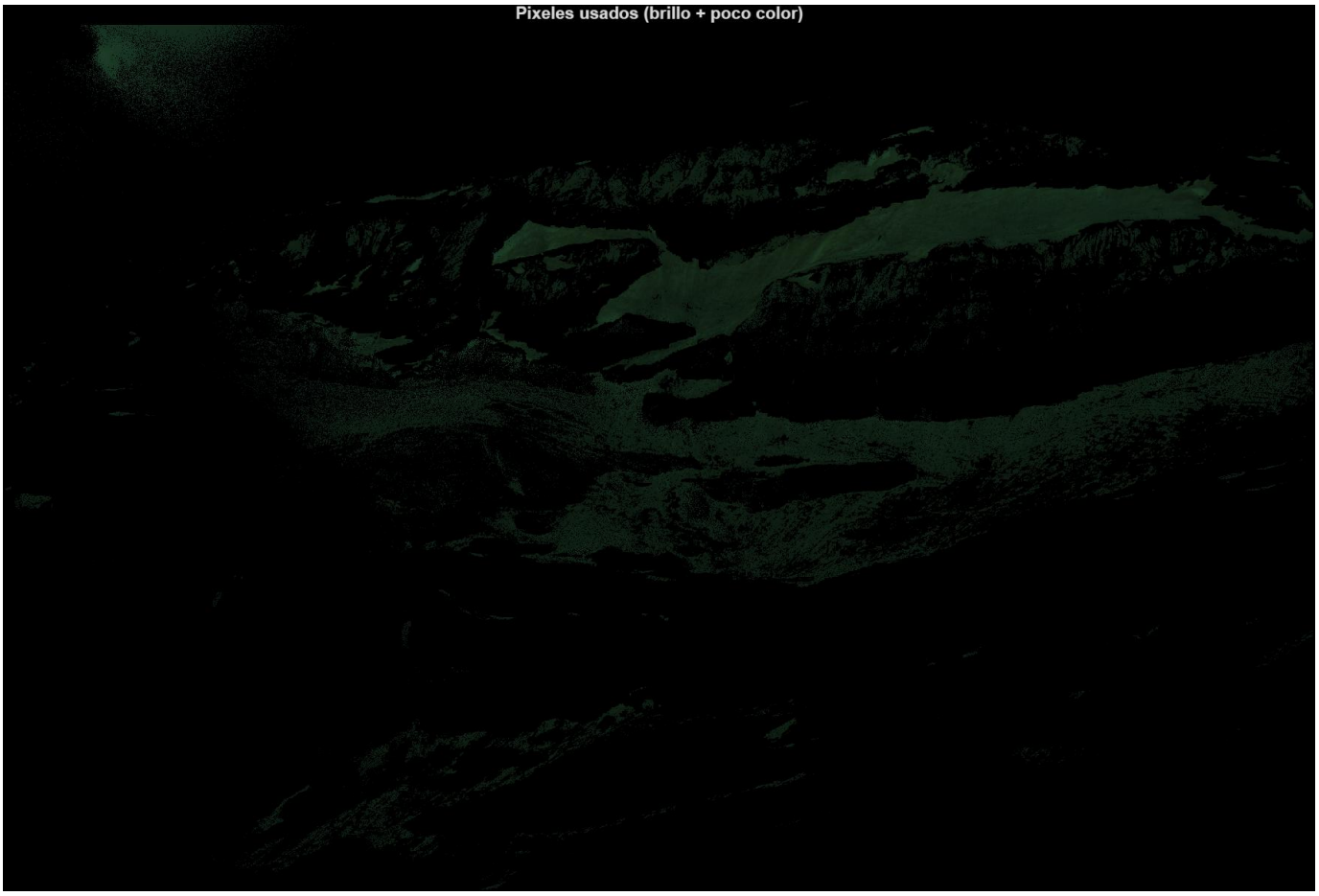
Medias: R=0.0594, G=0.1482, B=0.1068
Factores WB alternativo: fR=2.4943, fB=1.3880

Adjuntar imagen resultado.



Adjuntad código usado y una imagen donde se vean los puntos que cumplen esta condición.

```
umbral = 0.66 * max(Y(:));
col = (abs(I) + abs(Q)) ./ (Y + eps);

% Nueva máscara
umbral2 = col < 0.33;

usar2 = (Y > umbral) & umbral2;

%  píxeles usados
rgb_usar2 = cat(3, R .* usar2, G .* usar2, B .* usar2);
figure; imshow(rgb_usar2);
title('Pixeles usados (brillo + poco color)');
```



Volcad los nuevos factores de corrección.
Factores WB mejorado: fR = 2.0517, fB = 1.6033

Adjuntad la imagen resultante, valores usados de F y d y porcentaje de píxeles saturados.

F = 1.165;
delta = 0.03;

Pixeles saturados por debajo de 0: 0.4829 %
Pixeles saturados por encima de 1: 0.5110 %

3.5) Almacenamiento:

¿Qué factor de compresión se alcanza si usamos factores de calidad Q = 98, 95, 90?
Hemos obtenido lo siguiente:
JPEG Q=98: 2021664 bytes (2.022 MB) -> factor de compresion = 6.034
JPEG Q=95: 1237235 bytes (1.237 MB) -> factor de compresion = 9.859
JPEG Q=90: 796321 bytes (0.796 MB) -> factor de compresion = 15.318

Al disminuir la calidad, el tamaño del fichero se reduce y el factor de compresión aumenta.

El código que hemos usado en este apartado es el siguiente:

```
% 3.5) Almacenamiento

% Volvemos a una imagen de bytes (8 bits por canal)
im_out = uint8(255 * rgb_final);
% Guardamos versión sin perdidas
imwrite(im_out, 'foto.tif');
% Guardado JPEG con distintas calidades
   calidades = [98, 95, 90];
   info_tif = dir('foto.tif');
   tam_sin_comprimir = info_tif.bytes;
```

```
    for Q = calidades
        nombre_jpg = ['foto' num2str(Q) '.jpg'];
        imwrite(im_out, nombre_jpg, 'Quality', Q);

        info_jpg = dir(nombre_jpg);
        factor_compresion = info_tif.bytes / info_jpg.bytes;
        % Mostramos resultados
        fprintf('JPEG Q=%d: %d bytes (%.3f MB) -> factor de compresion = %.3f\n',
...
            Q, info_jpg.bytes, info_jpg.bytes / 1e6, factor_compresion);
    end
```